Flash Sale Checkout System

Technical Case Study Report

Campus Placement Technical Case Study
Role: Full-Stack Software Engineer

Submitted by: Shakthevell M
Email: shakthevell.m@gmail.com
Repository: https://github.com/Shakthe/goml
Date: 27 October 2026

Contents

1. Summary
2. Problem Analysis
3. Solution Architecture
4. Frontend Strategy
5. Backend and Middleware Architecture
6. Database Consistency and Concurrency Control
7. API Reference
8. Testing and Evidence
9. Application Screenshots
10. Requirement Traceability
11. Limitations and Production Next Steps
12. How to Run
Appendix A. Repository Structure
Appendix B. Evidence Index

1. Summary

1.1 The task

An e-commerce platform is running a flash sale of 500 smartphones at exactly 8:00 PM during a festival. More than 100,000 checkout requests are expected in the first 10 seconds. In earlier trial runs the platform suffered high latency, database row locks and request timeouts, and it oversold: 540 orders were confirmed for 500 phones. The task is to re-design the client and the backend so that exactly 500 units are sold, nothing is oversold, and users get a smooth experience under heavy load.

1.2 What was delivered

A complete, working system that runs with one command (`docker compose up`), together with automated tests that prove it meets the requirements:

- **Storefront and ops page** (React API built on Preact, Vite, Zustand): server-synced countdown, a buy button that locks on the first click, live stock over Server-Sent Events, order status polling, and a live operations view.
- **NGINX gateway**: serves the static frontend with long-lived cache headers, rate-limits each client IP, blocks known bot user agents, and load-balances across four API replicas.
- **Stateless checkout API** (Node.js 22, TypeScript, Fastify): reserves stock with one atomic Redis Lua script, publishes the order to RabbitMQ, and replies `202 Accepted` with a status URL.
- **Order workers**: consume the queue one message at a time and write each order to PostgreSQL in a single transaction with the conditional decrement `UPDATE ... WHERE stock > 0`.
- **Oversell Lab**: the original buggy checkout and two database-locking alternatives, kept side by side so the comparison asked for in the case study is measured instead of only described.
- **Tests**: REST Assured (API), k6 (load), Playwright (browser), Vitest (unit) and Lighthouse (performance).

1.3 Key results

All measurements were taken on one laptop (Windows 11, 8 logical cores, Docker Desktop) running the entire stack and the load generator at the same time.

Requirement	Target	Measured result	Status
No overselling	Exactly 500 sold	500 orders confirmed out of 100,000 requests; stock 0 in Redis and PostgreSQL	Met
Handle the 8 PM burst	100,000 requests in about 10 s	100,000 checkout requests completed in 7.2 s	Met
0% dropped requests	Every request answered	100,000 of 100,000 answered; 0 timeouts; 0 server errors (5xx)	Met

Requirement	Target	Measured result	Status
Low latency	No timeouts; p95 and p99.9 < 100 ms	Median 4.7 ms, p95 18.5 ms, p99.9 65.2 ms; 0 timeouts. Slowest single request 828 ms (see 8.4)	Met
Rate-limit bad actors	Bots blocked	97.4% of requests from a single-IP bot received 429	Met
Bundle size	< 200 KB	47.5 KB raw, 18.0 KB gzipped (JS + CSS)	Met
Time to Interactive	< 1.5 s	1.4 s on Lighthouse mobile (throttled), 0.3 s on desktop	Met
Load spread over replicas	Even distribution	25,004 to 25,010 requests per API replica (4 replicas)	Met
Automated tests	All pass	REST Assured 18/18, Playwright 5/5, Vitest 6/6, RabbitMQ restart test, k6 all thresholds in 3 runs	Met

2. Problem Analysis

2.1 Why the old system oversold

The previous checkout read the stock, checked it in application code and then wrote the new value. Under concurrency, two requests can read the same value before either writes. Both decide that a unit is available, both write the same new value and both insert an order. The table below shows the interleaving.

Time	Request A	Request B	Stock in database
t1	SELECT stock -> 37		37
t2		SELECT stock -> 37	37
t3	UPDATE stock = 36; INSERT order		36
t4		UPDATE stock = 36; INSERT order	36
Result	Two orders confirmed		Stock reduced by one only

This is a lost-update race condition. It is not fixed by adding application-level counters, because those counters suffer from the same race.

2.2 Why it was slow and timed out

- **Row lock contention.** Every request updated the same inventory row. Each update holds a row lock until its transaction ends, so thousands of requests queue behind one lock and hold database connections while waiting. The connection pool is exhausted and requests time out.
- **Synchronous database work in the request.** The HTTP request waited for the database for every buyer, including the 99.5% who could never get a phone because stock was already gone.
- **N+1 queries.** Loading the product, the stock and the user with separate queries per request multiplies database round trips under load.

2.3 Requirements from the case study

Layer	Challenge	Target
Frontend / Client	Prevent API spamming from rapid clicks; handle loading and failure states without freezing	Bundle < 200 KB; Time to Interactive < 1.5 s
API Gateway and Routing	Ingest massive parallel traffic without drops; block bots; distribute load across replicas	Rate-limit bad actors; 0% dropped requests
Application Logic	Decouple heavy database writes from the request-response cycle; handle high concurrency	Stateless clusters; event-loop safety
Database and Storage	Guarantee exactly 500 sold; avoid application-level double counters and row lock timeouts	ACID compliance; atomic decrements

3. Solution Architecture

3.1 Overview

The design follows two principles. First, deciding who gets a unit must be a single atomic step, so no two buyers can take the same unit. Second, the database must be taken out of the request path, so 100,000 requests do not become 100,000 transactions on one row. Redis decides who gets a unit, RabbitMQ carries the accepted orders, and workers write them to PostgreSQL at a steady rate. Only the requests that actually won a unit ever reach the database.

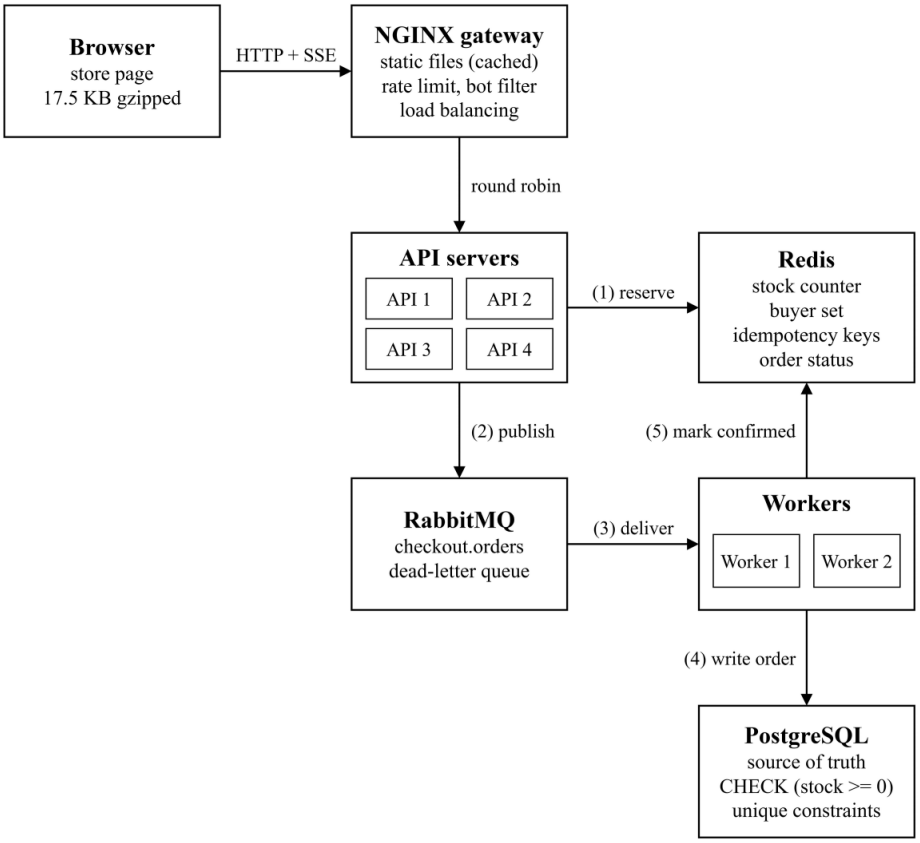


Figure 1: System architecture (source: report/architecture.drawio)

The numbers on the arrows are the order of events for one successful checkout:

(1) The API reserves a unit in Redis with one atomic Lua script. The script checks stock, the buyer and the idempotency key, then decrements the stock.

(2) The API publishes the order to RabbitMQ, waits for the broker to confirm it, and replies 202 Accepted with a status URL.

(3) Workers take one message at a time and acknowledge it only after the database commit.

(4) Each order is written to PostgreSQL in one transaction with UPDATE ... WHERE stock > 0.

(5) The worker marks the order CONFIRMED in Redis, where the browser's status polling reads it.

3.2 Components

Component	Role	How it scales
Browser app	Countdown, buy button, live stock, order status	Static files cached at the edge
NGINX gateway	Serves static files (CDN stand-in), per-IP rate limits, bot filter, round-robin load balancing	More gateway instances behind a cloud load balancer
API (4 replicas)	Verifies the request, runs the Redis reserve script, publishes to the queue, replies 202	Add replicas; no state is held in the process
Redis	Stock counter, buyer set, idempotency keys, order status, sale settings	Replica with Sentinel, or Redis Cluster (keys use hash tags)
RabbitMQ	Durable queue between "accepted" and "written to the database"; dead-letter queue	Quorum queues on a 3-node cluster
Workers (2)	Read one message at a time; write the order in one transaction	Add workers; database writes are bounded by stock, not traffic
PostgreSQL	Source of truth for stock and orders; constraints that forbid overselling	Off the hot path; one primary is sufficient

3.3 Checkout request, step by step

Step	Component	Action
1	Browser	Sends POST /api/checkout with a signed session token and an Idempotency-Key header. The buy button is locked from this moment.
2	NGINX	Refuses known bot user agents (403) and IPs over the rate limit (429). Forwards everything else to any API replica.
3	API	Verifies the token and that the sale has started. If this replica already knows stock is 0, it answers 409 from memory without calling Redis.
4	Redis	Runs the reserve Lua script as one atomic step: repeated key returns the same order; a second order for the same customer is refused; stock 0 is refused; otherwise stock is decremented and the reservation is recorded.
5	RabbitMQ	The API publishes the order as a persistent message and waits for the broker confirmation. If publishing fails, the reservation is released and the customer gets 503 with Retry-After.
6	API	Replies 202 Accepted with Location: /api/orders/{id}. Work done in the request: one HMAC check, one Redis call, one publish.
7	Worker	In one transaction: INSERT the order (ON CONFLICT DO NOTHING), then UPDATE inventory SET stock = stock - 1 WHERE product_id = $1 AND stock > 0. Commits, sets the order to CONFIRMED in Redis and acknowledges the message.
8	Browser	Polls the status URL with exponential backoff and jitter (400 ms doubling to 4 s) and shows "Order confirmed".

3.4 Technology stack

Area	Choice	Reason
Frontend	React API on Preact (compat), Vite, TypeScript, Zustand	Familiar React code at about one fifth of the size of React DOM
Gateway	NGINX 1.27	Load balancing, rate limiting and static file serving in one proven component
API and workers	Node.js 22, TypeScript, Fastify 5	Non-blocking I/O suits many small concurrent requests; Fastify has low overhead

Area	Choice	Reason
Cache / inventory tokens	Redis 7.2 (append-only file enabled)	Single-threaded execution makes Lua scripts atomic
Message queue	RabbitMQ 3.13	Per-message acknowledgement, redelivery, dead-letter queue, publisher confirms
Database	PostgreSQL 16	ACID transactions, CHECK and UNIQUE constraints
Packaging	Docker Compose	Whole system (4 API, 2 workers) starts with one command
Testing	REST Assured, JUnit 5, k6, Playwright, Vitest, Lighthouse	API, load, browser, unit and performance coverage

4. Frontend Strategy

4.1 Preventing repeated requests

Three layers stop a user from sending more than one order, even by accident:

1. **State machine.** The buy button is driven by a small state machine (`web/src/checkout-machine.ts`). The first click moves it to `submitting`; any click in that state, or after a final answer, is ignored and the button is disabled.
2. **Synchronous lock.** A React ref is set before the request starts, so two clicks that land in the same frame (before the screen re-renders) still produce one request. The Playwright test fires five clicks in one frame and confirms exactly one request is sent.
3. **Idempotency key.** Each purchase attempt carries an `Idempotency-Key` header. If a request is retried after a network error, it reuses the same key and the server returns the original order instead of creating a second one.

Debouncing alone was not used because it delays the first click. Locking on the first click is both faster for the user and stricter.

4.2 Sale status updates: polling, WebSockets or Server-Sent Events

Option	Strengths	Weaknesses	Decision
Polling	Simple; works everywhere	Many wasted requests; updates arrive late; 100,000 browsers polling adds load at the worst moment	Used as fallback, and for order status
WebSockets	Two-way, low latency	A second protocol to balance and secure; the browser never needs to send on this channel	Not used
Server-Sent Events	One-way push over plain HTTP; automatic reconnect; passes through NGINX and CDNs	One-way only (sufficient here)	Used for live stock and sale state

Each API replica reads Redis once every 500 ms and pushes to its connected browsers only when the value changed. 100,000 connected browsers therefore cost each replica two Redis reads per second, not 100,000. If the stream fails three times in a row, the page switches to polling every 2 to 3 seconds with random jitter.

4.3 Optimistic UI and loading / failure states

As soon as the server returns 202, the page tells the user a unit is reserved, before the worker has written the order. Every state has a clear message and the page never blocks:

State	Button text	Message shown
Before the sale	Opens at {time}	Countdown (hours, minutes, seconds)
Sale open	Buy now · ₹49,999	-
Submitting	Placing order…	Sending your order…
Accepted (202)	Confirming…	A unit is reserved for you. Confirming your order {id}…

State	Button text	Message shown
Confirmed	Order confirmed	Done. Your order {id} is confirmed.
Sold out	Sold out	Sorry, all units are sold.
Already ordered	Already ordered	You already have an order in this sale {id}. One per customer.
Rate limited (429)	Try again	Too many attempts. Wait a few seconds, then try again.
Error / network failure	Try again	Retry reuses the same idempotency key, so it is safe.

4.4 State management and clock

Zustand (about 1 KB) holds the session, the sale view, the server clock offset, the live-feed state and the checkout state. The countdown uses the server time measured when the page loads (corrected for half the round-trip time), so a customer whose phone clock is wrong still sees the correct countdown, and the button unlocks when the server says the sale has started.

4.5 Asset delivery and pre-fetching

- Built files have content hashes in their names and are served with `Cache-Control: public, max-age=31536000, immutable`; `index.html` is served with `no-cache`. This is exactly what a CDN (CloudFront or Cloudflare) caches at the edge; in this project NGINX plays that role.
- The product image is preloaded in the HTML head so it downloads in parallel with the JavaScript. Vite adds `modulepreload` links for the app code.
- The ops page is a separate code chunk that shoppers never download; it is prefetched only when someone hovers over its link.
- Gzip compression is enabled at the gateway.

4.6 Bundle size and Time to Interactive

With React DOM the first build was 240 KB raw (76 KB gzipped): under the budget when gzipped, but over it when measured raw. The same code built with Preact's compatibility layer is 47.5 KB raw and 18.0 KB gzipped, which meets the budget under either reading. The build fails automatically if the budget is exceeded (`web/scripts/check-bundle.mjs`).

Build output (evidence/unit/web-build-bundle-budget.txt)

```
index-6ypcXpFH.css                          6.7 KB     2.1 KB gz
index-DgEHKYwG.js                          36.3 KB    14.1 KB gz
Ops-BYtGjagg.js                             4.5 KB     1.9 KB gz
total: 47.5 KB raw, 18.0 KB gzipped (budget 200 KB raw)
```

5. Backend and Middleware Architecture

5.1 Gateway: load balancing, rate limiting and bot filtering

NGINX receives all traffic. Docker DNS returns every API replica for the name `api`, and NGINX distributes requests round robin with keep-alive connections. Each client IP may make 2 checkout requests per second with a burst of 5; anything beyond that receives 429 immediately. Requests with an empty user agent or a known scraping library are refused with 403.

Excerpt from infra/nginx/nginx.conf

```
limit_req_zone  $binary_remote_addr zone=checkout:20m rate=2r/s;
limit_conn_zone $binary_remote_addr zone=perip:20m;
map $http_user_agent $bad_bot {
  default 0;  ""  1;
  ~*(python-requests|scrapy|aiohttp|go-http-client|httpx|libwww-perl|headlesschrome) 1;
}
upstream api_pool { server api:3000 max_fails=0; keepalive 512; }
location = /api/checkout {
  if ($bad_bot) { return 403 '{"code":"BLOCKED", ...}'; }
  limit_req  zone=checkout burst=5 nodelay;
  limit_conn perip 20;
  proxy_pass http://api_pool;
}
```

A determined bot farm using many IP addresses and real browsers can get past simple rules. In production this layer is backed by the CDN's bot management (for example Cloudflare or AWS WAF Bot Control) and a sign-in requirement; the rules here are the in-house second line.

5.2 Stateless API and event-loop safety

- The API keeps nothing important in memory. Its cached view of the sale and its "sold out" flag are refreshed from Redis every 500 ms; losing them only costs one Redis read. Any replica can serve any request, which the `X-Served-By` response header shows.
- The request path does no blocking work: one HMAC verification, one Redis call (auto-pipelined by the ioredis client) and one queue publish. There is no synchronous file or network I/O.
- Live updates use one timer per replica, not one Redis read per connected browser.
- If a replica loses its RabbitMQ connection it exits and Docker restarts it, instead of continuing in a broken state.

5.3 Redis inventory tokens

Redis executes one command or script at a time on a single thread. A Lua script therefore runs as one atomic step: no other buyer's request can run between reading the stock and decrementing it. A plain `DECR` would also be atomic, but the script additionally enforces "one per customer" and idempotency in the same step, which a single command cannot do.

Reserve script (server/src/scripts.ts)

```
local existing = redis.call('HGET', KEYS[3], ARGV[3])        -- idempotency key used?
if existing then return {'DUPLICATE', existing} end
existing = redis.call('HGET', KEYS[2], ARGV[1])              -- customer already bought?
if existing then return {'ALREADY_PURCHASED', existing} end
```

```
local stock = tonumber(redis.call('GET', KEYS[1]) or '0')
if stock <= 0 then return {'SOLD_OUT', ''} end
redis.call('DECR', KEYS[1])
redis.call('HSET', KEYS[2], ARGV[1], ARGV[2])
redis.call('HSET', KEYS[3], ARGV[3], ARGV[2])
redis.call('HSET', KEYS[4], 'status', 'PENDING', 'userId', ARGV[1], 'createdAt', ARGV[4])
redis.call('EXPIRE', KEYS[4], tonumber(ARGV[5]))
return {'RESERVED', ARGV[2]}
```

All keys of one sale share the hash tag `{saleId}`, so the script also works on Redis Cluster. Redis runs with an append-only file (fsync every second) so the counter survives a restart.

5.4 Message queue and workers

- **Publisher confirms.** The API waits for RabbitMQ to confirm it has stored the message before replying 202. An accepted order is never lost silently.
- **Durable queue, persistent messages.** Orders survive a broker restart.
- **Manual acknowledgement after commit.** A worker acknowledges a message only after the database transaction commits. If it crashes in between, RabbitMQ redelivers the message, and the `INSERT ... ON CONFLICT DO NOTHING` makes the second attempt a no-op, so stock is never taken twice.
- **Sequential consumption.** Each worker has prefetch 1, so it processes one order at a time, as in the reference solution.
- **Dead-letter queue.** A message that fails twice (for example the database is down) goes to `checkout.orders.dead` for inspection; its count is shown on the ops page.
- **RabbitMQ rather than Kafka.** The flow is one message per unit sold (500 messages) and needs per-message acknowledgement and redelivery, which RabbitMQ provides directly. Kafka is better suited to high-volume event streams read by several independent consumers; it could be added later for "order confirmed" events.

5.5 Every request gets an answer

"0% dropped requests" is interpreted as: every request receives a correct and fast response. It does not mean every request gets a phone.

Situation	Response	Answered by
Unit reserved	202 Accepted + Location header	Redis
Same Idempotency-Key again	202, same order	Redis
Sold out	409 SOLD_OUT	API memory once stock is known to be 0
Customer already ordered	409 ALREADY_PURCHASED + link to the order	Redis
Sale not started	425 NOT_STARTED + start time	API memory
Too many requests from one IP	429	NGINX
Known bot user agent	403 BLOCKED	NGINX
Missing or forged session	401	API
Queue unavailable	503 + Retry-After (reservation released)	API

6. Database Consistency and Concurrency Control

6.1 Comparison of approaches

Approach	How it works	Correct?	Behaviour at flash-sale scale
SELECT then UPDATE (old code)	Read stock, decide in the application, write the computed value	No	Lost updates; oversells
Pessimistic locking	SELECT ... FOR UPDATE locks the row until commit	Yes	Every buyer waits in line for one row lock while holding a connection; leads to lock timeouts and pool exhaustion
Optimistic locking	Read a version number; UPDATE only if it is unchanged; retry on conflict	Yes	When everyone updates the same row almost every attempt conflicts; buyers are turned away while stock remains
Atomic Redis script + queue (this design)	Check and decrement in one atomic step in memory; write to the database asynchronously	Yes	Decision in under a millisecond inside Redis; the database sees only 500 writes

These are not only theoretical differences. Section 8.5 measures all four approaches under the same load.

6.2 The worker transaction

The database write uses exactly the conditional statement from the reference solution, with bound parameters. The order is inserted first, so a redelivered message cannot take stock a second time.

Worker transaction (server/src/worker.ts)

```
BEGIN;
INSERT INTO orders (id, sale_id, product_id, user_id, idempotency_key, status, strategy)
VALUES ($1, $2, $3, $4, $5, 'CONFIRMED', 'queue')
ON CONFLICT DO NOTHING RETURNING id;           -- 0 rows: already processed, ROLLBACK

UPDATE inventory SET stock = stock - 1
WHERE product_id = $1 AND stock > 0;            -- 0 rows: no stock, ROLLBACK and release
COMMIT;
```

6.3 Database constraints

Redis is the fast gate, but PostgreSQL is the source of truth and has the final say. Even if Redis were wrong, the following constraints make overselling or double orders impossible:

Excerpt from infra/db/init.sql

```
CREATE TABLE inventory (
  product_id  text PRIMARY KEY REFERENCES products (id),
  stock       integer NOT NULL CHECK (stock >= 0),
  version     integer NOT NULL DEFAULT 0      -- used only by the optimistic lab strategy
);
CREATE TABLE orders (
  id uuid PRIMARY KEY, sale_id text NOT NULL, product_id text NOT NULL,
  user_id text NOT NULL, idempotency_key text NOT NULL, status text NOT NULL, ...,
  CONSTRAINT one_order_per_user_per_sale    UNIQUE (sale_id, user_id),
  CONSTRAINT one_order_per_idempotency_key  UNIQUE (sale_id, idempotency_key)
);
```

6.4 Failure handling

Failure	What happens
Double click or network retry	Same Idempotency-Key; the script returns the same order (DUPLICATE). No second unit is taken.
Same customer in a new tab	New key, but the customer already has an order: 409 ALREADY_PURCHASED with a link to it.
Queue publish fails	The release script returns the unit to stock and removes the hold; the customer gets 503 and can retry with the same key.
Worker crashes after commit, before acknowledgement	RabbitMQ redelivers; INSERT ... ON CONFLICT DO NOTHING finds the order, so stock is not taken twice.
Database or Redis error in the worker	Message requeued once; on a second failure it moves to the dead-letter queue.
Database refuses the decrement	Transaction rolled back, the Redis hold released, order marked FAILED. This is the last safety net and should not occur.
API replica loses RabbitMQ	The replica exits and is restarted; other replicas keep serving.
Redis restarts	Data restored from the append-only file (fsync every second).

7. API Reference

Method and path	Purpose	Responses
POST /api/session	Issue an anonymous signed session token	201
GET /api/sale	Sale details, stock left, server time, state	200
GET /api/sale/stream	Server-Sent Events stream of sale state	200 (event stream)
POST /api/checkout	Place an order. Headers: Authorization: Bearer {token}, Idempotency-Key	202, 400, 401, 403, 409, 425, 429, 503
GET /api/orders/{id}	Order status; visible only to the customer who placed it	200, 401, 404
GET /api/stats	Aggregate counts for the ops page (no customer data)	200
POST /api/admin/sale	Start a new sale (stock, start time, strategy). Header: X-Admin-Token	200, 400, 401
GET /api/health	Liveness check	200

Example exchange (from the smoke test; token shortened)

```
POST /api/checkout
Authorization: Bearer 3f0c...e1.Vg9...
Idempotency-Key: 9c1d4e7a2b5f4e0c8a6d3b1e7f2a9c4d

HTTP/1.1 202 Accepted
Location: /api/orders/cf3e825d-9b8e-4db9-9f1d-5a024a50af18
{"orderId":"cf3e825d-9b8e-4db9-9f1d-5a024a50af18","status":"PENDING",
 "statusUrl":"/api/orders/cf3e825d-9b8e-4db9-9f1d-5a024a50af18"}

GET /api/orders/cf3e825d-9b8e-4db9-9f1d-5a024a50af18   ->
{"orderId":"cf3e...","status":"CONFIRMED"}
```

8. Testing and Evidence

8.1 Test environment

Item	Value
Machine	Laptop, Windows 11 Pro, 8 logical CPU cores, Docker Desktop 29.6.1, Docker Compose 5.1.4
System under test	NGINX 1.27, 4 API replicas, 2 workers (Node.js 22), Redis 7.2, RabbitMQ 3.13, PostgreSQL 16
Load generator	k6 (grafana/k6 container) on the same Docker network, sharing the same CPU
API tests	REST Assured 5.5.0, JUnit 5.11.3, Awaitility 4.2.2, Maven 3.9 on Java 17 (container)
Browser tests	Playwright 1.49 with Google Chrome
Performance	Lighthouse 12.8.2
Date of runs	27 October 2026, between 21:30 and 22:00 IST

All tests run against the real, running system. There are no mocks in the API, load or browser tests.

8.2 Summary of results

Suite	Tool	Tests	Result	Evidence file
API behaviour, gateway, concurrency	REST Assured + JUnit 5	18	18 passed, 0 failed	evidence/api-tests/
100,000-request burst	k6	3 runs, 9 thresholds	All thresholds passed in all 3 runs	evidence/k6/flash-sale-run*.txt
RabbitMQ restart	Shell script	1	Passed (failed before the worker fix)	evidence/resilience/
Strategy comparison	k6	4 runs	Completed (table in 8.5)	evidence/k6/lab-results.md
Browser flows	Playwright	5	5 passed	evidence/e2e/
Buy-button logic	Vitest	6	6 passed	evidence/unit/vitest-run.txt
Code quality	ESLint	-	0 problems	evidence/unit/eslint-run.txt
Bundle budget	Build script	-	47.5 KB raw (budget 200 KB)	evidence/unit/web-build-bundle-budget.txt
Page performance	Lighthouse	2 profiles	Score 100 on both	evidence/lighthouse/

8.3 API tests (REST Assured)

The suite runs in a Maven container on the same Docker network as the system (`docker compose --profile test run --rm api-tests`). Each request carries its own simulated client IP so that only the rate-limit test is affected by rate limiting. Each test starts a fresh sale through the admin endpoint.

Class	Test	What it verifies	Result
CheckoutApiTest	saleView	Sale endpoint returns product, stock, start time and server time	Pass
	tooEarly	Checkout before the sale opens returns 425 with the start time	Pass
	acceptedThenConfirmed	202 with Location header; the worker confirms the order; Redis and database stock both drop to 9 of 10	Pass
	idempotentRetry	Same Idempotency-Key twice returns the same order; only one unit is taken	Pass
	onePerCustomer	Second checkout by the same customer with a new key returns 409 ALREADY_PURCHASED	Pass
	soldOut	With 1 unit, the second buyer gets 409 SOLD_OUT	Pass
	validation	Missing or forged token gives 401; a bad Idempotency-Key gives 400	Pass
	orderPrivacy	Order visible to its owner (200), not to another customer (404) or anonymous caller (401)	Pass
	adminAuth	Admin endpoint requires the token (401) and validates input (400)	Pass
	clientErrorsStayClientErrors	Unsupported media type gives 415 and malformed JSON gives 400, never 500	Pass
GatewayTest	blocksBots	python-requests, Go-http-client and Scrapy user agents receive 403	Pass
	rateLimitsPerIp	20 rapid requests from one IP produce 429s; other IPs are unaffected	Pass
	loadBalanced	40 requests reach at least two different API replicas	Pass
	staticAssetsCached	index.html is no-cache; hashed assets are immutable	Pass
ConcurrencyTest	queueSellsExactlyTheStock	1,000 buyers released at the same instant for 50 units: exactly 50 x 202, 950 x 409; then 50 orders, stock 0 in both stores, empty queue, no dead letters	Pass
	naiveOversells	The original SELECT-then-UPDATE path oversells (reproduces the bug)	Pass
	pessimisticIsCorrect	SELECT ... FOR UPDATE sells exactly 50	Pass
	optimisticIsCorrect	Version-checked updates never sell more than 50	Pass

Console output (evidence/api-tests/rest-assured-run.txt)

```
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 25.11 s -- in
sale.CheckoutApiTest
[optimistic] 1000 buyers, 50 units -> responses {200=50, 409=950}
[optimistic] confirmed orders: 50 for 50 units
[naive] 1000 buyers, 50 units -> responses {200=1000}
[naive] confirmed orders: 1000 for 50 units
[queue] 1000 buyers, 50 units -> responses {202=50, 409=950}
[pessimistic] 1000 buyers, 50 units -> responses {200=50, 409=950}
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 87.49 s -- in
sale.ConcurrencyTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.668 s -- in
sale.GatewayTest
[INFO] Tests run: 18, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time:  01:59 min
[INFO] Finished at: 2026-10-27T16:08:57Z
```

8.4 Load test (k6): the 8 PM burst

Script: `tests/load/flash-sale.js`. A sale of 500 units is started, then two scenarios run at the same time:

- **buyers**: 100,000 checkout requests shared among 100 virtual users, so 100 requests are in flight at any moment. Every request is a different customer with a different client IP and its own idempotency key.
- **bot**: 100 requests per second for 10 seconds from a single IP address, reusing one customer.

After the load, the script waits for the queue to drain and checks the final state through `/api/stats`. The run fails if any threshold is missed:

Threshold	Required	Measured	Result
Requests answered	100%	100.00% (100,000 of 100,000)	Pass
Server errors (5xx)	0	0	Pass
Requests with no response (timeouts, connection errors)	0	0	Pass
Buyer latency p95	< 100 ms	18.5 ms	Pass
Buyer latency p99.9	< 100 ms	65.2 ms	Pass
Bot requests rate-limited	> 90%	97.40%	Pass
Confirmed orders after drain	Exactly 500	500	Pass
Stock in PostgreSQL / Redis	0 / 0	0 / 0	Pass
Queue depth / dead letters	0 / 0	0 / 0	Pass

Metric	Value
Checkout requests sent and answered	100,000 in 7.2 s (about 13,900 per second during the burst)
Responses to buyers	499 x 202 accepted, 99,501 x 409 sold out. The 500th unit went to the bot's single customer account; its repeated requests returned that same order, and it could not buy a second unit.
Latency (all checkout requests)	median 4.6 ms, p90 13.6 ms, p95 18.4 ms, p99 32.4 ms, p99.9 65.1 ms; slowest single request 828 ms
Requests per API replica	25,004 / 25,006 / 25,006 / 25,010
Orders written per worker	249 and 251

Console output (evidence/k6/flash-sale-run.txt)

```
     scenarios: (100.00%) 2 scenarios, 120 max VUs, 2m0s max duration (incl. graceful
stop):
              * bot: 100.00 iterations/s for 10s (maxVUs: 20, exec: bot, gracefulStop: 30s)
              * buyers: 100000 iterations shared among 100 VUs (maxDuration: 1m30s, exec:
buyer, gracefulStop: 30s)
buyers ✓ [ 100% ] 100 VUs    0m07.2s/1m30s   100000/100000  shared iters
       ✓ exactly 500 confirmed orders
       ✓ database stock is 0
       ✓ redis stock is 0
       ✓ queue drained, nothing dead-lettered
   ✓ bot_requests_blocked...........:  97.40%  975 out of 1001
     checkout_accepted_202..........: 499     39.593671/s
   ✓ checkout_answered..............:  100.00% 100000 out of 100000
   ✓ checkout_no_response...........:  0       0/s
```

```
   ✓ checkout_server_errors_5xx.....:  0       0/s
     checkout_sold_out_409..........: 99501   7895.009721/s
     http_req_duration..............: avg=6.68ms   min=120.68µs med=4.63ms  max=827.61ms
p(90)=13.6ms  p(95)=18.42ms p(99)=32.42ms p(99.9)=65.07ms
     http_req_failed................: 0.00%   0 out of 101004
     http_reqs......................: 101004  8014.26681/s
     vus_max........................: 120     min=120              max=120
```

About the numbers. The first attempt used a fixed arrival rate of 10,000 new requests per second. On this laptop k6 itself could not generate that rate while sharing the CPU with the system, and it reported iterations it never started; the server still answered every request it received. The test was therefore changed to send exactly 100,000 requests as fast as possible. A second run, made while a browser was taking screenshots of the ops page, completed 100,000 requests in 14.1 s with the same correctness results (`evidence/k6/flash-sale-run-while-screenshotting.txt`). Production capacity would be measured with the load generator on separate machines.

Requests in flight. The first version of this test used 1,200 virtual users, so 1,200 requests were in flight at all times. The machine can only complete about 11,000 requests per second, so the extra requests simply waited in line: median 55 ms, p95 191 ms, slowest 1.03 s, for the same 100,000 requests in 8.8 s. The same burst was repeated with fewer requests in flight. Throughput stayed the same and latency dropped sharply, so the test now uses 100, and its latency thresholds were tightened from p95 < 1 s to p95 < 100 ms and p99.9 < 100 ms. Every run below had exactly 500 orders, every request answered and no errors. The 400, 200 and 100 rows were measured before the RabbitMQ change described below.

Requests in flight	Median	p90	p95	Slowest request	Evidence file (evidence/k6/)
1,200	55.6 ms	144.9 ms	191.6 ms	1,028 ms	flash-sale-summary-1200vus.json
400	28.8 ms	96.5 ms	139.4 ms	2,188 ms	tuning-vus400-summary.json
200	9.4 ms	28.2 ms	36.6 ms	194 ms	tuning-vus200-summary.json
100	5.0 ms	14.7 ms	19.2 ms	112 ms	tuning-vus100-summary.json

Three runs. The final configuration was run three times back to back. Run 1 was chosen in advance as the one shown above. All three passed every threshold with exactly 500 orders:

Run	Burst	Median	p95	p99	p99.9	Slowest request
1	7.2 s	4.7 ms	18.5 ms	32.4 ms	65.2 ms	828 ms
2	7.1 s	4.9 ms	17.8 ms	29.9 ms	49.6 ms	134 ms
3	7.7 s	5.3 ms	19.8 ms	32.2 ms	58.5 ms	132 ms

The slowest request. 99.9% of requests finish in under 66 ms in every run, but the single slowest request ranges from about 130 ms to 828 ms. About 25 diagnostic runs that logged every request over 50 ms showed that every slow request was a "sold out" answer served from the API's memory, a path that takes about 5 ms. With Node's garbage-collection trace on, the longest pause in any API process during a burst was 10 ms. The slow requests come in clusters of 50 to 300 ms, sometimes on one API replica and sometimes on all of them at once: moments when the operating system pauses a process while about 20 busy processes share 8 cores.

Freshly started API processes are also slower until their code is compiled. One unrecorded warm-up run right after the APIs restarted had p99.9 at 122 ms, and run 1 came right after an API replica rejoined the load balancer, which most likely explains its 828 ms.

One change helped and was kept: RabbitMQ's Erlang schedulers were spinning while idle and using 1 to 1.6 cores, and are now told not to. This also made the burst faster (about 7.5 s instead of 9 s). Tried and not kept, because they made no clear difference or made it worse: 4 NGINX workers instead of 8, a larger Node young-generation heap, less frequent garbage collection in k6, higher CPU priority for the API, NGINX and Redis, and pinning containers to separate cores. Keeping the slowest request reliably under 100 ms needs the load generator on a separate machine.

The ops-page screenshots below were taken during a separate 100,000-request run with Chrome capturing the page at the same time (`evidence/k6/flash-sale-run-while-screenshotting.txt`). With the browser also using CPU, that run took 10.8 s and its p99.9 was 102 ms, missing the 100 ms threshold by 2 ms; it still had exactly 500 orders and stock 0 in Redis and PostgreSQL.

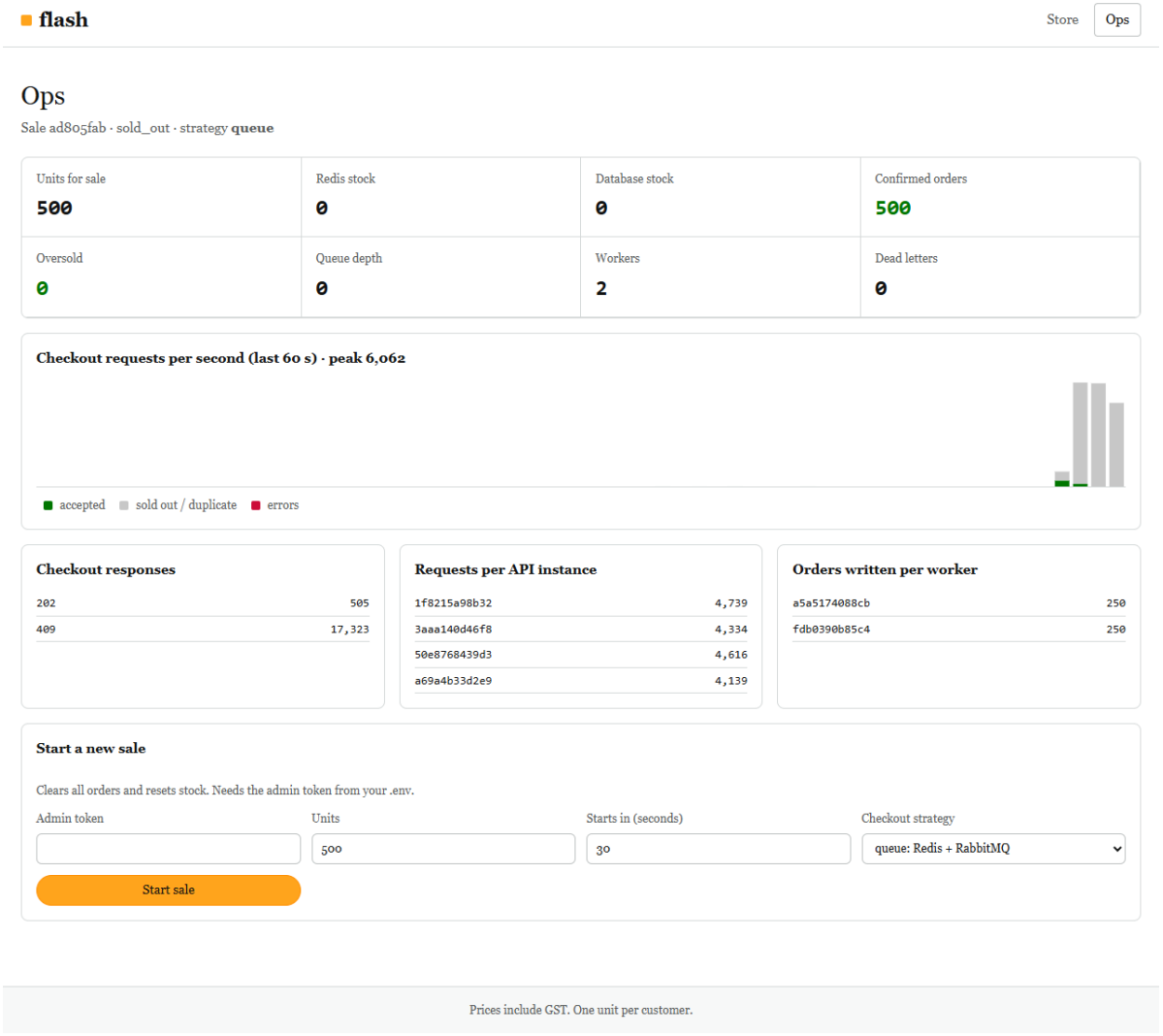


Figure 2: Ops page a few seconds into the 100,000-request run: about 17,800 checkout responses so far, all 500 units already confirmed in the database, Redis and database stock both 0, queue empty, nothing oversold

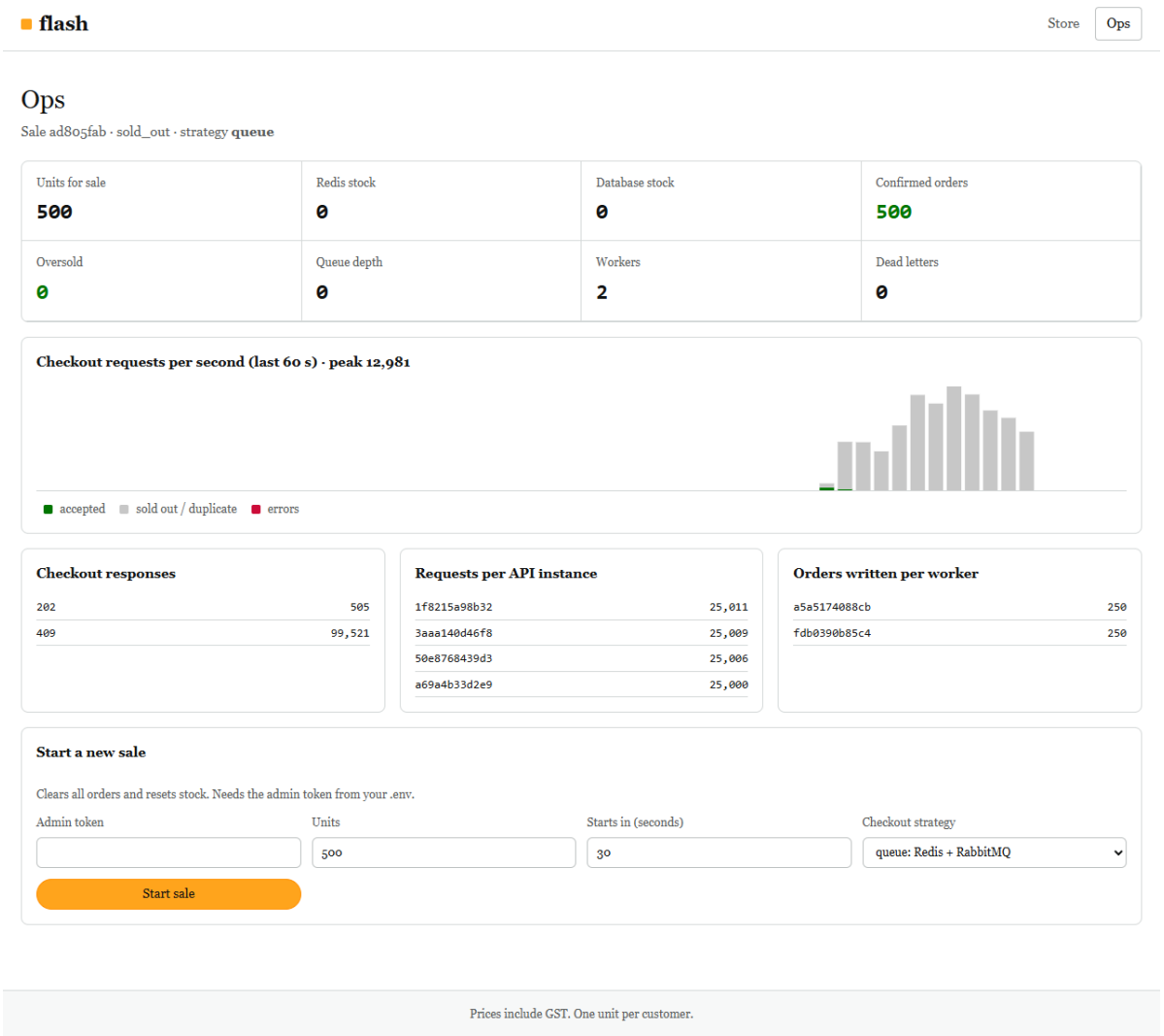


Figure 3: Ops page after the same run: peak 12,981 requests per second, 500 confirmed orders, no dead letters, requests spread evenly over four API replicas (25,000 to 25,011 each)

8.5 Oversell Lab: four strategies under the same load

Script: `tests/load/lab.js`, run for each strategy by `tests/load/run-lab.sh`. Each run: 5,000 buyers, 500 units, 200 concurrent clients. The table is generated from the k6 output files.

Strategy	Orders for 500 units	Oversold	Turned away (conflict)	Median	p95	Max	Req/s
Naive SELECT then UPDATE	5,000	+4,500	0	266.7 ms	448.0 ms	697 ms	547
Pessimistic (FOR UPDATE)	500	0	0	259.2 ms	468.1 ms	1,304 ms	547
Optimistic (version, 3 retries)	167	0	4,833	261.4 ms	412.7 ms	632 ms	523
Redis Lua + queue (this design)	500	0	0	18.8 ms	74.3 ms	146 ms	1,174

- **Naive**: reproduces the original bug at a much larger scale: 5,000 orders for 500 phones. Most requests read a stale stock value and wrote back "value minus one".

- **Pessimistic**: correct, but every buyer waits in line for the row lock; the worst request took 1.3 s even at this small scale.
- **Optimistic**: never oversells, but it is unfair under contention: 4,833 buyers were told to try again while 333 units were still unsold.
- **Redis + queue**: correct, about 14 times faster at the median and the only strategy whose cost does not grow with lock contention.

The REST Assured `ConcurrencyTest` repeats this with 1,000 buyers for 50 units and asserts the results; its console output is in section 8.3.

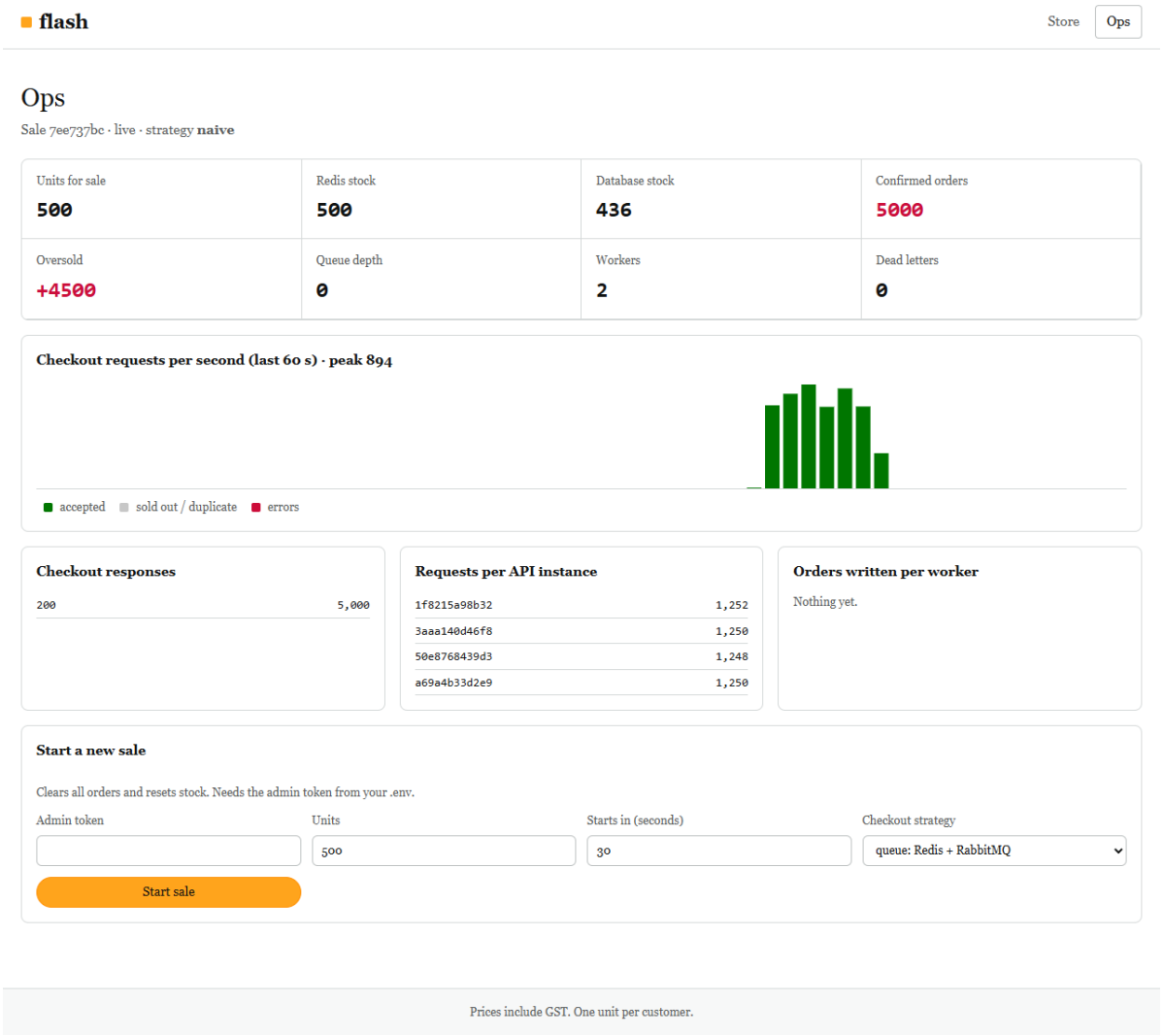


Figure 4: Ops page after a run with the naive strategy: 5,000 orders confirmed for 500 units (+4,500 oversold) while the database still shows 436 in stock, the lost-update bug. Redis shows 500 because the naive path does not use Redis.

8.6 Browser tests (Playwright)

Test	What it verifies	Result
Before 8 PM	Countdown visible, buy button disabled, live feed connected (desktop and mobile)	Pass
Sale opens	The button becomes enabled by itself when the start time passes, without a reload	Pass

Test	What it verifies	Result
Rapid clicks	Five clicks in one frame send exactly one /api/checkout request; the order is confirmed; stock shows 499; after a reload the same customer is told they already have an order	Pass
Last unit	One buyer gets the last unit; the other shopper's page switches to "Sold out" by itself through SSE	Pass
Ops page	Confirmed orders, Redis stock and database stock agree	Pass

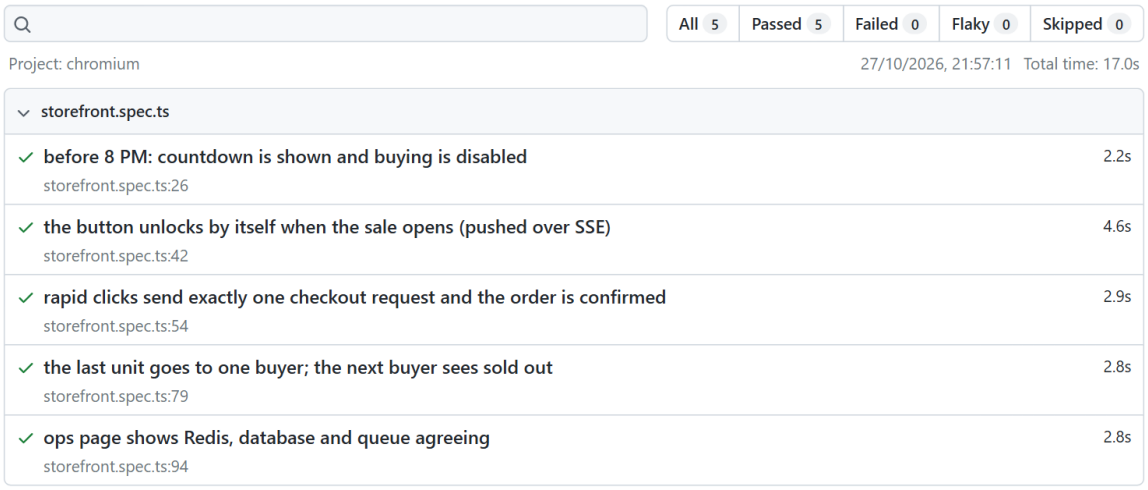


Figure 5: Playwright HTML report: 5 passed, 0 failed (evidence/e2e/report)

8.7 Unit tests, lint and bundle budget

Vitest (evidence/unit/vitest-run.txt)

```
✓ src/checkout-machine.test.ts  (6 tests)  4ms
Test Files  1 passed (1)
     Tests  6 passed (6)
```

The six tests cover: the button locks after the first click; 202 leads to "queued" and then "confirmed"; 409, 425 and 429 map to the right states; a retry after failure reuses the original idempotency key; a network error is retryable; and no further purchase is possible after a final result. ESLint reported no problems (exit code 0).

8.8 Frontend performance (Lighthouse)

Profile	Score	FCP	LCP	Time to Interactive	TBT	CLS	Transferred
Mobile (4x CPU slowdown, 150 ms RTT, 1.6 Mbps)	100	1.1 s	1.4 s	1.4 s	0 ms	0.051	42 KiB
Desktop	100	0.3 s	0.3 s	0.3 s	0 ms	0	42 KiB

Time to Interactive is taken from the Lighthouse JSON reports (`interactive` audit: 1,433 ms on mobile, 327 ms on desktop). Lighthouse 12 no longer displays this metric in its visual report, so the figures below show the other metrics. Measured on the final page, including the 24 KB product image.

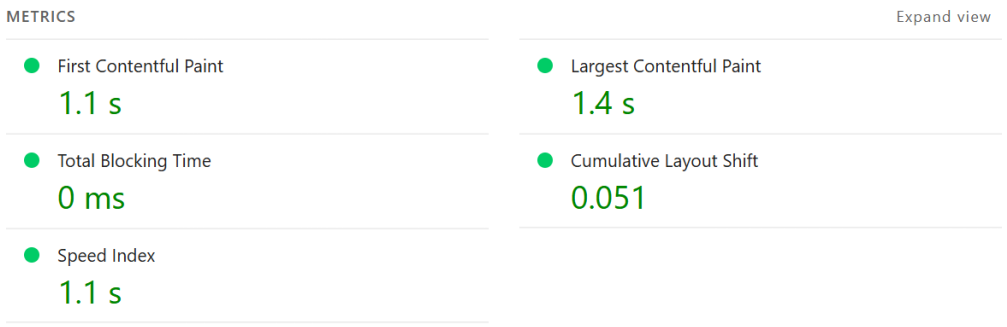


Figure 6: Lighthouse metrics, mobile profile (evidence/lighthouse/mobile.report.html)

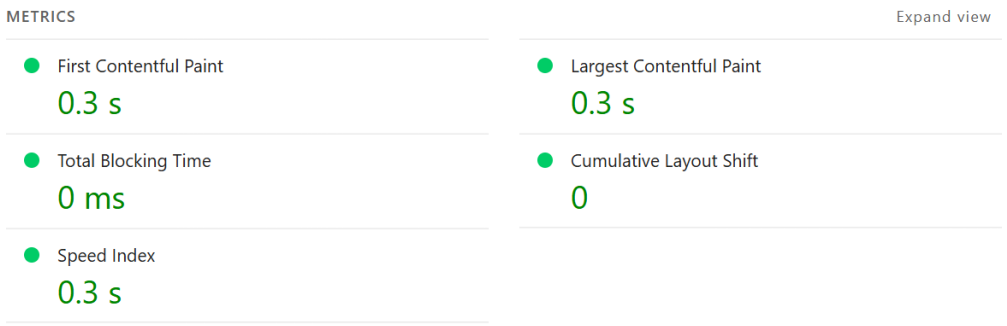


Figure 7: Lighthouse metrics, desktop profile (evidence/lighthouse/desktop.report.html)

8.9 Defects found during testing

#	Defect	Found by	Fix
1	Client errors raised by the web framework (for example 415 Unsupported Media Type) were returned as 500	REST Assured	Error handler now passes 4xx through; regression test clientErrorsStayClientErrors added
2	NGINX failed to start because of a duplicated proxy_read_timeout directive	First start of the stack	Directive moved out of the shared include
3	The button always said "Opens at 8:00 PM", even for a sale scheduled at another time	Review of Playwright screenshots	Label now uses the actual start time
4	First build with React DOM was 240 KB raw, over the 200 KB budget	Bundle budget check	Built with Preact compat: 45.3 KB raw
5	After a RabbitMQ restart the workers stayed up but consumed nothing, so orders stayed in the queue	Latency diagnostics; then the RabbitMQ restart test (0 consumers)	Worker exits on a lost connection and Docker restarts it; test tests/resilience/rabbitmq-restart.sh added to CI
6	NGINX resolves the api name once at startup; an API container that restarts on a new IP gets no traffic until the gateway restarts. No requests fail, but load spreads over fewer replicas	k6 run showing 3 replicas instead of 4	Not fixed yet. Fix: resolver 127.0.0.11 and resolve on the upstream server

Test-side issues were also corrected during development: the API tests needed a JSON library to send request bodies, and one test wrongly assumed text/plain was an unsupported media type (it is supported, so the test now uses XML).

9. Application Screenshots

The screenshots in this section show the final UI: a plain white product page laid out like a large online store, with the product image, the details and a buy box. They were captured by a script that goes through the same flows as the Playwright tests (`evidence/screenshots/`).

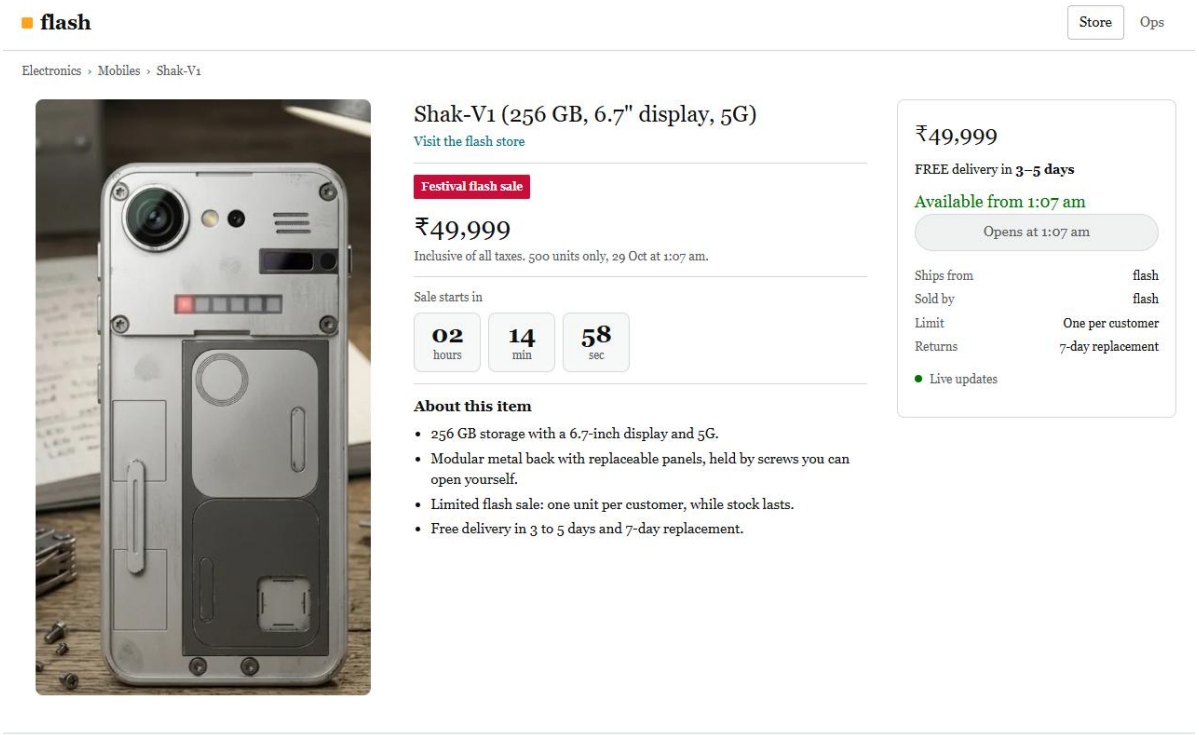


Figure 8: Before the sale: server-synced countdown, buy button disabled

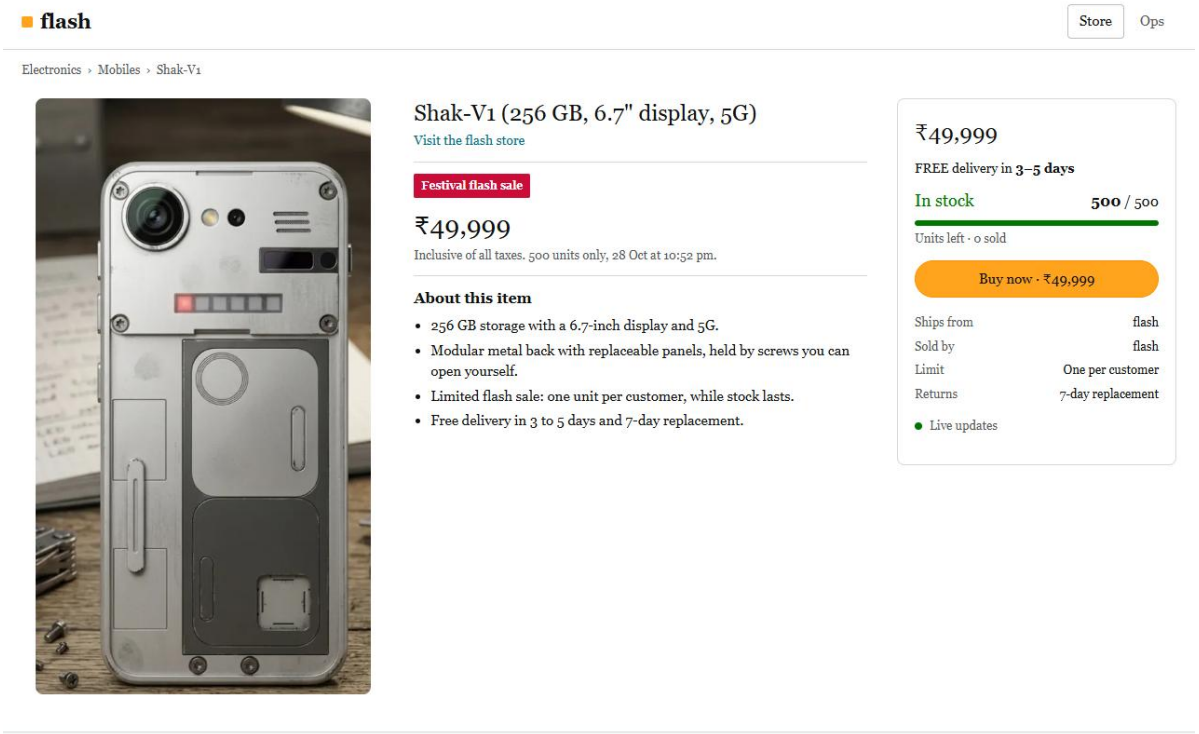


Figure 9: Sale open: stock bar and enabled buy button

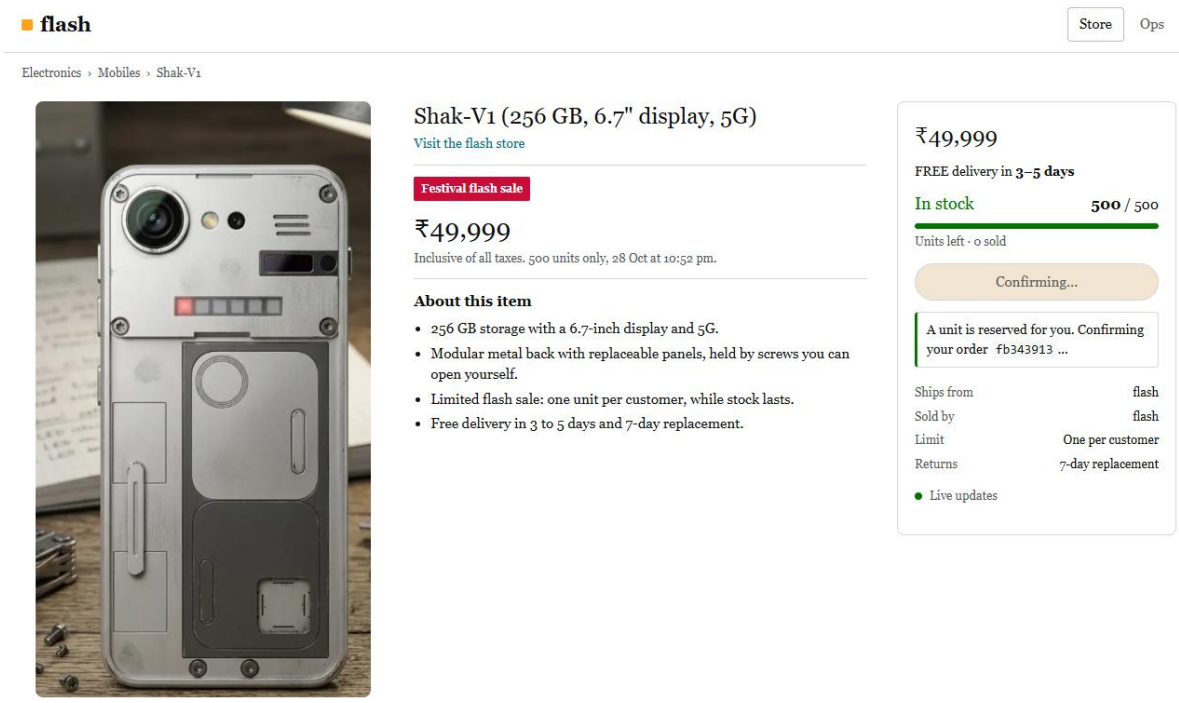


Figure 10: Right after the click: button locked, a unit reserved, the order being confirmed by the worker

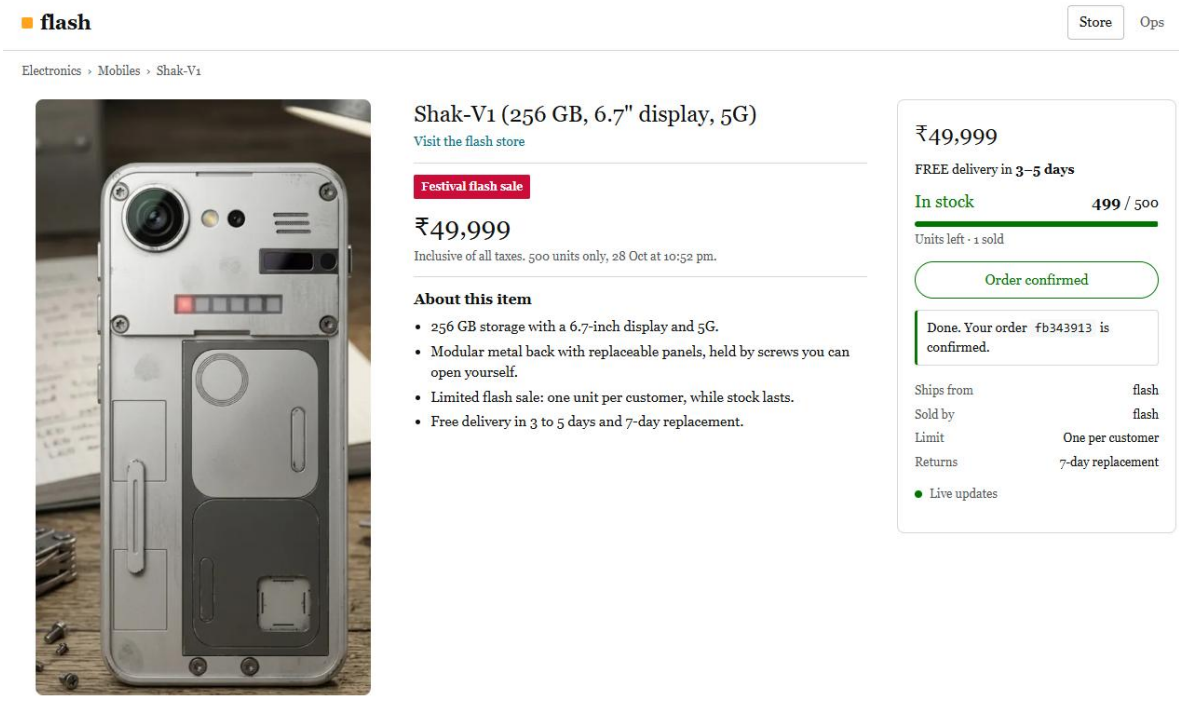


Figure 11: Order confirmed by the worker; stock updated live to 499

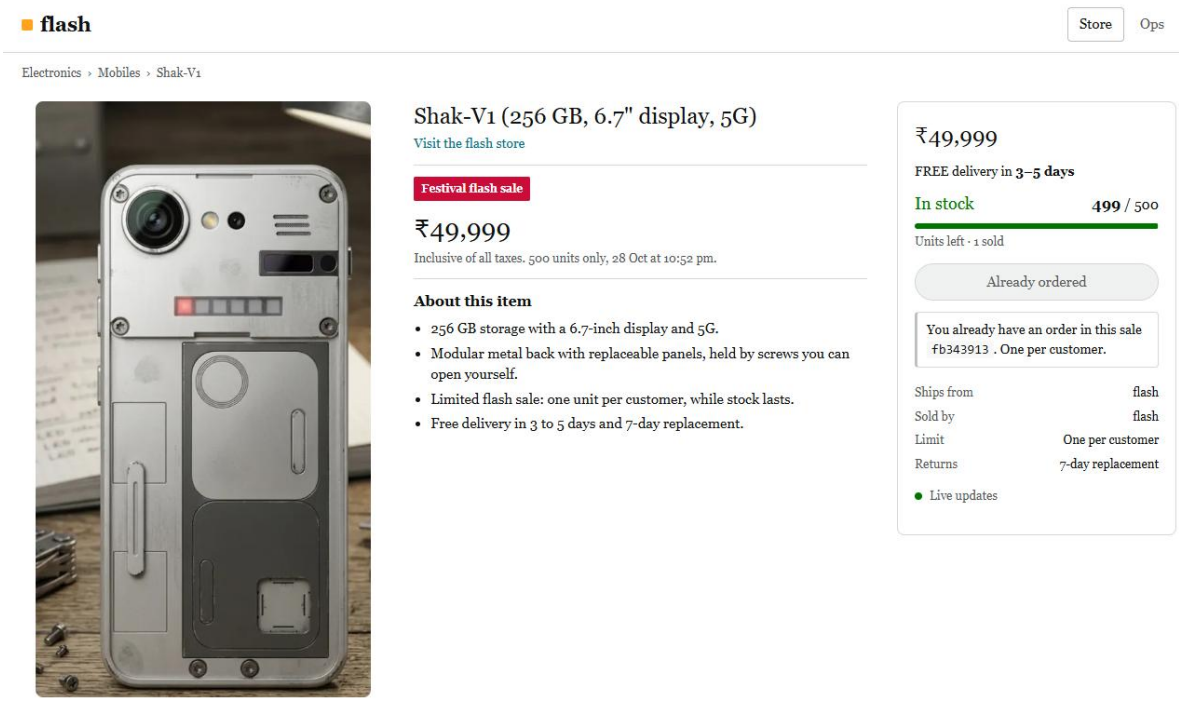


Figure 12: Same customer after a page reload: one unit per customer

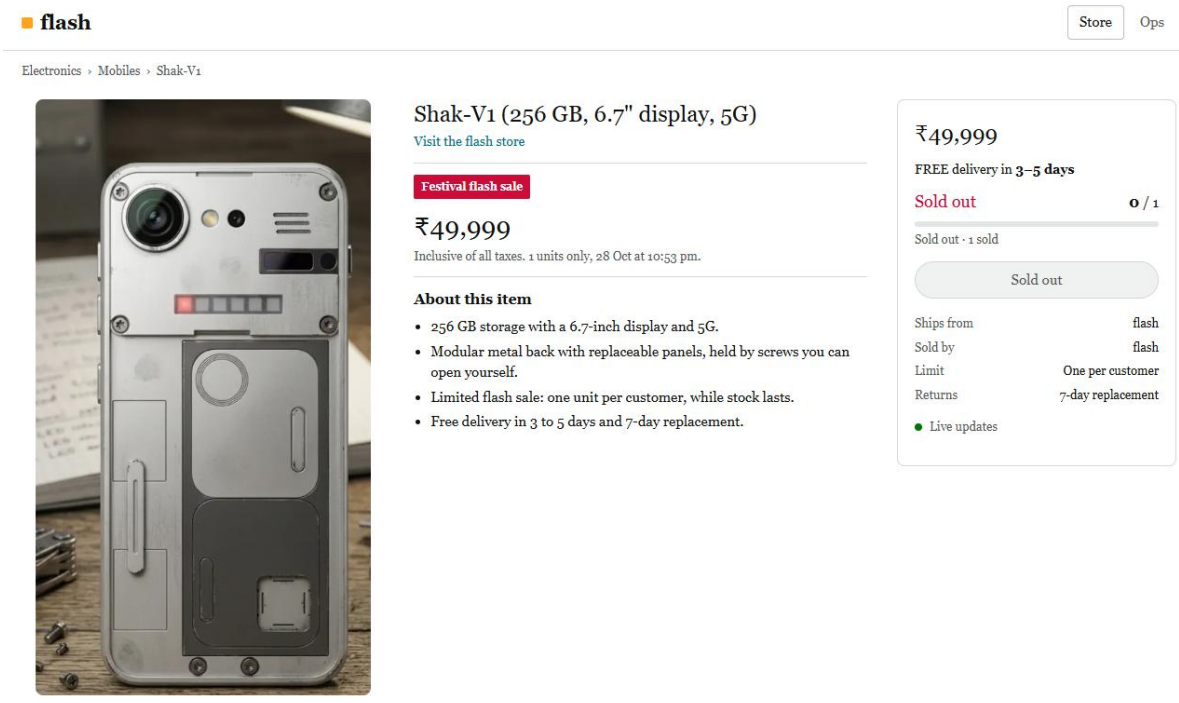


Figure 13: Another shopper after the last unit was sold: page switched to "Sold out" without a reload

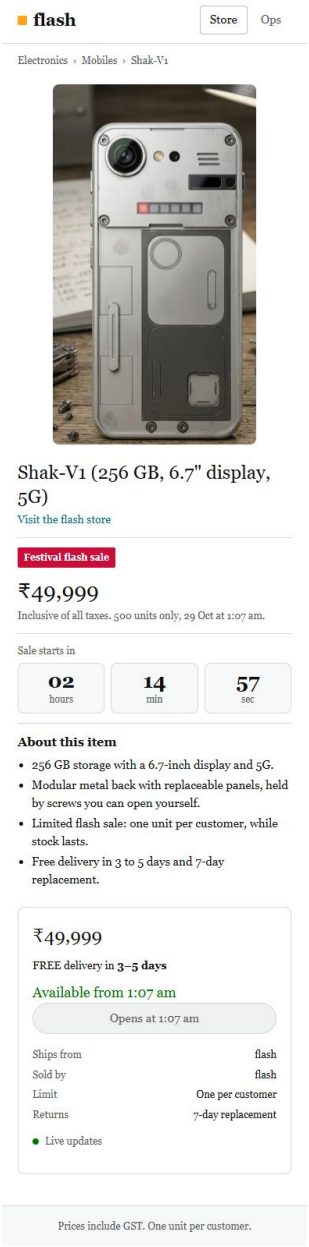


Figure 14: Mobile layout (390 px wide)

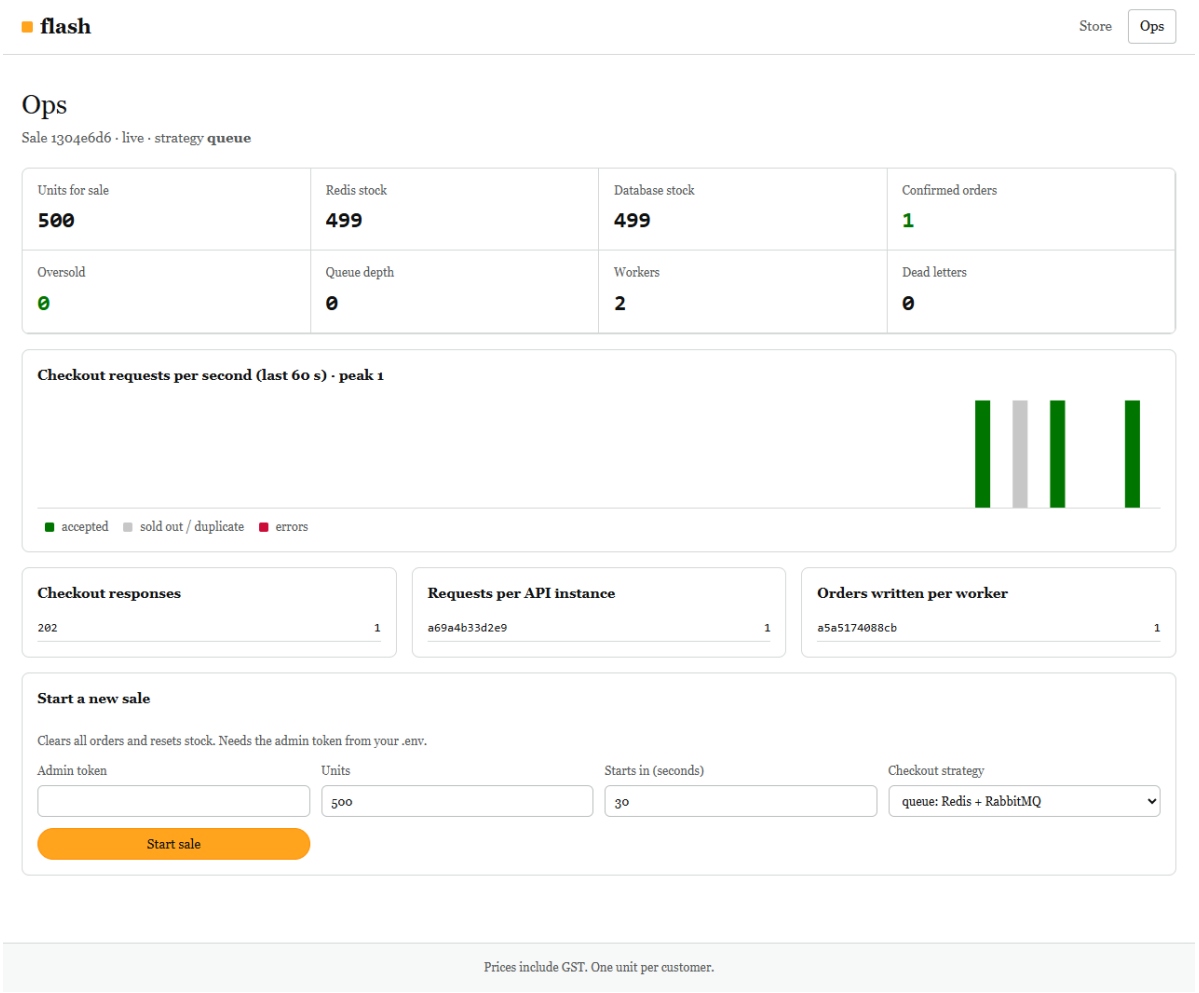


Figure 15: Ops page after one order: Redis stock, database stock and confirmed orders agree

10. Requirement Traceability

10.1 Technical core requirements

Requirement	Implementation	Evidence
Prevent API spamming from rapid clicks	Button state machine, synchronous ref lock, Idempotency-Key	Playwright "rapid clicks"; REST Assured idempotentRetry; Vitest
Graceful loading and failure states	Status message for every state; retry with same key; SSE to polling fallback	Screenshots, section 9
Bundle < 200 KB	Preact compat, code splitting, build-time budget check	47.5 KB raw / 18.0 KB gzipped
Time to Interactive < 1.5 s	Small bundle, preloaded image, no blocking work	Lighthouse mobile 1.4 s, desktop 0.3 s
Ingest parallel traffic without drops	NGINX keep-alive upstream; in-memory sold-out path	100,000 / 100,000 answered
Block bot traffic; rate-limit bad actors	Per-IP limit_req and limit_conn; user-agent filter	k6 bot 97.5% limited; GatewayTest
Distribute load across replicas	NGINX round robin over 4 replicas	25,004 to 25,010 requests per replica
Decouple database writes from the request	RabbitMQ queue, workers, 202 + Location	acceptedThenConfirmed
Stateless clusters	No state in the API process	Any replica serves any request
Event-loop safety	No blocking work; one Redis call per request; timer-based broadcast	p95 18.5 ms, p99.9 65.2 ms at about 13,900 requests/s
ACID compliance, atomic decrements	Lua DECR; transactional conditional UPDATE; CHECK and UNIQUE constraints	Exactly 500 orders; ConcurrencyTest
No application-level double counters	Redis is the single counter; PostgreSQL updated in the same transaction as the order	Redis stock equals database stock after every run

10.2 Evaluation framework

Topic	Where it is covered
UI debouncing / throttling	Section 4.1
Asset pre-fetching and CDN	Section 4.5
Dynamic polling versus WebSockets	Section 4.2
Optimistic UI updates	Section 4.3
Load balancing	Section 5.1
Distributed cache (Redis inventory tokens)	Section 5.3
Message queue and workers	Section 5.4
Pessimistic versus optimistic locking	Sections 6.1 and 8.5
Why atomic Redis operations are safer than SELECT then UPDATE	Sections 2.1, 5.3 and 8.5

10.3 Model reference solution

Reference step	Status in this project
A1. Lock the UI on the first click	Done (section 4.1)
A2. Serve static assets from CDN edge locations	Immutable caching of hashed assets; NGINX as CDN stand-in; production uses CloudFront or Cloudflare
A3. State manager; wait times through polling or SSE	Zustand, SSE and backoff polling
B1. NGINX or ALB in front of stateless servers	NGINX in front of 4 stateless API replicas
B2. Redis inventory countdown with atomic decrements	Lua reserve script
B3. Message queue; 202 Accepted with a polling URL	RabbitMQ; Location: /api/orders/{id}
C1. Consumers read the queue sequentially	Workers with prefetch 1 and acknowledgement after commit
C2. UPDATE inventory SET stock = stock - 1 WHERE product_id = :id AND stock > 0	Exactly this statement, with bound parameters
C3. No race condition, no overselling	Proven by k6 and REST Assured

10.4 Grading scorecard ("Exemplary" tier)

Criterion	Where it is addressed
Identifies the N+1 and database row-lock bottleneck	Section 2.2; measured in section 8.5
Asynchronous queueing combined with atomic Redis tokens	Sections 3, 5.3 and 5.4
Awareness of UI bundle optimisation and edge CDNs	Sections 4.5 and 4.6

11. Limitations and Production Next Steps

11.1 What is simplified

- **Payment** is out of scope. A confirmed order holds a unit for the customer. In production the worker would create a payment intent; an unpaid hold would expire and return the unit (the release script already does this for failed orders).
- **Sign-in** is an anonymous signed session per browser, which is enough to enforce one unit per customer.
- **CDN**: NGINX serves the static files with the same cache headers a CDN would use; no real CDN is configured.
- **Single nodes**: Redis and RabbitMQ run as single instances.
- **One machine for everything**: the load generator shares 8 cores with the system it tests. The 99.9th percentile is stable, but the single slowest request (about 130 ms to 828 ms) reflects operating-system scheduling on this laptop rather than the application.
- **NGINX DNS**: the gateway resolves the API replicas once at startup, so a replica that restarts on a new IP is left out until the gateway restarts (defect 6 in section 8.9).
- **Load figures** come from one laptop that also ran the load generator, so they show correctness and relative performance, not production capacity.

11.2 Production next steps

- CloudFront or Cloudflare in front, with WAF and bot management; trust X-Forwarded-For only from the CDN address ranges.
- API replicas in an auto-scaling group or Kubernetes deployment behind a cloud load balancer.
- Redis with a replica and Sentinel (or a managed service); RabbitMQ quorum queues on three nodes.
- Metrics to Prometheus and Grafana instead of the Redis counters behind the ops page; error reporting to Sentry or Rollbar.
- A virtual waiting room only if traffic grows by another order of magnitude; sold-out answers are already served from memory.

12. How to Run

```
git clone https://github.com/Shakthe/goml.git && cd goml
docker compose up -d --build

# Store:  http://localhost:8080        Ops page: http://localhost:8080/ops
# To start a sale now: on /ops enter the admin token dev-admin-token

# API tests (REST Assured)
docker compose --profile test run --rm api-tests

# Load test: 100,000 requests (on Git Bash for Windows prefix with MSYS_NO_PATHCONV=1)
docker compose --profile test run --rm k6 run /scripts/flash-sale.js

# Strategy comparison
bash tests/load/run-lab.sh

# Order pipeline survives a RabbitMQ restart
bash tests/resilience/rabbitmq-restart.sh

# Browser tests
cd tests/e2e && npm ci && npx playwright install chromium && npx playwright test

# Unit tests and bundle budget
cd web && npm ci && npm test && npm run build
```

Appendix A. Repository Structure

```
server/        API and worker (Node.js 22, TypeScript, Fastify, ioredis, amqplib, pg)
  src/api.ts            HTTP routes, SSE, checkout, stats, admin
  src/worker.ts         queue consumer and order transaction
  src/scripts.ts        Redis Lua reserve and release scripts
  src/db-strategies.ts  naive, pessimistic and optimistic lab strategies
web/          Storefront and ops page (React API on Preact, Vite, Zustand)
  src/checkout-machine.ts  buy-button state machine
  src/sale-feed.ts         SSE with polling fallback
  scripts/check-bundle.mjs bundle budget check
infra/nginx/  gateway configuration and image
infra/db/     PostgreSQL schema and seed data
tests/api/    REST Assured tests (Maven, run in Docker)
tests/load/   k6 scripts (flash-sale.js, lab.js, run-lab.sh)
tests/e2e/    Playwright tests and screenshot script
tests/resilience/  RabbitMQ restart test
docs/         architecture, decisions, requirement checklist, testing
evidence/     outputs and screenshots from the test runs
docker-compose.yml  whole system: 4 API, 2 workers, Redis, RabbitMQ, PostgreSQL, NGINX
```

Appendix B. Evidence Index

File	Contents
evidence/api-tests/rest-assured-run.txt	Full Maven console output of the REST Assured run
evidence/api-tests/surefire/	JUnit XML and text reports per test class
evidence/k6/flash-sale-run.txt	k6 console output, official 100,000-request run (100 requests in flight)
evidence/k6/flash-sale-summary.json	k6 machine-readable summary of the same run
evidence/k6/flash-sale-run-2.txt, -3.txt and summary-2/3.json	Runs 2 and 3 of the final configuration
evidence/k6/flash-sale-run-1200vus.txt, flash-sale-summary-1200vus.json	Earlier 100,000-request run with 1,200 requests in flight
evidence/resilience/rabbitmq-restart-run.txt	Output of the RabbitMQ restart test
evidence/k6/tuning-vus400/200/100-summary.json	Same burst with 400, 200 and 100 requests in flight
evidence/k6/flash-sale-run-while-screenshotting.txt	100,000-request run with 1,200 in flight, taken while capturing the ops screenshots
evidence/k6/lab-*.txt, lab-*.json	k6 output for each Oversell Lab strategy
evidence/k6/lab-results.md	Generated strategy comparison table
evidence/e2e/playwright-run.txt, report/	Playwright console output and HTML report
evidence/unit/	Vitest, ESLint and bundle-budget outputs
evidence/lighthouse/	Lighthouse HTML and JSON reports (mobile and desktop)
evidence/screenshots/	Application and ops-page screenshots